

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a71e6a04d0bb0e5a6.jsonl`  
- SHA-256: `da4d51b8a184898495d0997cac1d7c7a30718437432dece32bac5cff6e6bdfa7`  
- Source modified: `2026-06-10T06:13:10+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:11:16.113Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("eng-eval") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"run_regression.py is labeled a CI gate but is not in CI and structurally cannot gate CI","severity":"high","area":"regression gate","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/run_regression.py (+ .github/workflows/ci.yml, backend/eval/regression.py:30,117-118)","evidence":".github/workflows/ci.yml contains only an import-smoke job and run_all_tests.py ? no invocation of run_regression.py anywhere. DEFAULT_BASELINE_PATH = output/regression_baseline.json (regression.py:30) and the predictions DB both live under output/, which is gitignored (verified: git check-ignore matches .gitignore:39 'output/'). On a fresh checkout the gate either exits 2 (no DB) or, if a DB exists but no baseline, sets regressed=False with reason 'no baseline recorded' (regression.py:117-118) and exits 0 ? a silent pass. Additionally the baseline snapshot stores only {precision, recall, f1} keyed by video_id::stage (regression.py:63-64,75-78): iou_threshold, detector, tolerance, and the GT file identity/hash are not recorded, so re-running with --iou 0.3 against a baseline captured at 0.5 compares incommensurable numbers silently.", "why_it_matters":"The docstring, CODE_MAP (\$1C '[CI gate]'), and exit-code semantics all present this as the CI regression gate. As implemented it is a local manual tool: nothing fails a PR when quality regresses. For Phase-0 (Gen-0 ship-gate 'beat heuristic LOVO 0.480'), a gate that can silently pass and cannot validate that baseline and run share an IoU threshold cannot be trusted as the ship decision mechanism. Fix needs: commit/artifact the baseline outside output/, refuse-or-warn-loudly on missing baseline (e.g. exit 3), and store iou/detector/GT-hash in the baseline entry."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:11:23.111Z)

```
1      """CLI: regression-test the production tool against golden-set ground truth.
2
3      Obtains ground truth from an annotation provider (a tier), scores the video's
4      stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5      detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7      Examples
8      -----
9      Snapshot the current numbers as the baseline (first run / after a real improvement):
10         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13         python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14     """
15     import os
16     import sys
17     import json
18     import argparse
19     import logging
20
21     from backend.infrastructure.database import Database
22     from backend.config import load_config
23     from backend.annotations import annotation_registry
24     from backend.eval import regression
25
26
27     def _default_db_path() -> str:
28         cfg = load_config()
29         return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32     def build_parser() -> argparse.ArgumentParser:
33         p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34         p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35         p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37         p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38         p.add_argument("--stage", choices=("candidates", "final"), default="final")
39         p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40         p.add_argument("--iou", type=float, default=0.5)
41         p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42         p.add_argument("--db", default=None)
43         p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44         p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46         p.add_argument("--json", dest="json_out", default=None)
47         return p
48
49
50     def main(argv=None) -> int:
51         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
```

```

52     args = build_parser().parse_args(argv)
53
54     db_path = args.db or _default_db_path()
55     if not os.path.exists(db_path):
56         print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
57         return 2
58     db = Database(db_path)
59
60     result = regression.regress_with_provider(
61         db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
62         stage=args.stage, iou_threshold=args.iou, detector=args.detector,
63         tolerance=args.tolerance, baseline_path=args.baseline,
64     )
65
66     print(regression.format_result(result))
67
68     if args.update_baseline:
69         regression.save_baseline(result.video_id, result.stage, result.precision,
70                                 result.recall, result.f1, path=args.baseline)
71         print(f"Baseline updated in {args.baseline}.")
72
73     if args.json_out:
74         with open(args.json_out, "w") as f:
75             json.dump(result.as_dict(), f, indent=2)
76             print(f"JSON written to {args.json_out}")
77
78     # Non-zero exit on regression so CI can gate on it.
79     return 1 if result.regressed else 0
80
81
82 if __name__ == "__main__":
83     raise SystemExit(main())
84

```

3. user (2026-06-10T06:11:24.044Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7         ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8         predictions  ? the production pipeline's stored DB rows   [backend.eval.harness]
9         comparison   ? interval matching + P/R/F1                 [backend.eval.metrics]
10        baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import logging
20    from dataclasses import dataclass, asdict
21    from typing import Dict, List, Optional
22
23    from backend.infrastructure.database import Database
24    from backend.eval import metrics
25    from backend.eval.harness import get_predictions
26    from backend.annotations import annotation_registry
27
28    logger = logging.getLogger(__name__)

```

```

29
30 DEFAULT_BASELINE_PATH = os.path.join("output", "regression_baseline.json")
31 DEFAULT_TOLERANCE = 0.05
32
33
34 @dataclass
35 class RegressionResult:
36     video_id: str
37     stage: str
38     iou_threshold: float
39     precision: float
40     recall: float
41     f1: float
42     # Baseline values compared against (None when no baseline existed yet).
43     baseline_precision: Optional[float]
44     baseline_recall: Optional[float]
45     tolerance: float
46     regressed: bool
47     reasons: List[str]
48
49     def as_dict(self) -> dict:
50         return asdict(self)
51
52
53 # ----- baseline store
54
55 def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
56     """Load the per-video baseline snapshot file (returns {} if absent)."""
57     if not os.path.isfile(path):
58         return {}
59     with open(path) as f:
60         return json.load(f)
61
62
63 def _baseline_key(video_id: str, stage: str) -> str:
64     return f"{video_id}::{stage}"
65
66
67 def save_baseline(video_id: str, stage: str, precision: float, recall: float,
68                  f1: float, path: str = DEFAULT_BASELINE_PATH) -> None:
69     """Snapshot a video/stage's current numbers as the new 'known good' baseline.
70
71     Use this to (re)baseline after a *legitimate* production improvement, so old
72     baselines don't raise false regressions. Stored as plain JSON for easy diffing.
73     """
74     baselines = load_baselines(path)
75     baselines[_baseline_key(video_id, stage)] = {
76         "video_id": video_id, "stage": stage,
77         "precision": precision, "recall": recall, "f1": f1,
78     }
79     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
80     with open(path, "w") as f:
81         json.dump(baselines, f, indent=2, sort_keys=True)
82     logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
83 _baseline_key(video_id, stage), precision, recall)
84
85 # ----- the runner
86
87 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
88            stage: str = "final", iou_threshold: float = 0.5,
89            detector: Optional[str] = None,
90            tolerance: float = DEFAULT_TOLERANCE,
91            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
92     """Score stored predictions for one video against ground truth and compare to

```

```

baseline.
93
94     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
95     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
96     precision OR recall falls more than `tolerance` below the snapshotted baseline.
97     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
98     caller can choose to snapshot the current numbers via `save_baseline`.
99     """
100     preds = get_predictions(db, video_id, stage, detector=detector)
101     s = metrics.score(preds, ground_truth, iou_threshold)
102
103     baselines = load_baselines(baseline_path)
104     b = baselines.get(_baseline_key(video_id, stage))
105
106     reasons: List[str] = []
107     regressed = False
108     base_p = base_r = None
109     if b is not None:
110         base_p, base_r = b["precision"], b["recall"]
111         if s.precision < base_p - tolerance:
112             regressed = True
113             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -
{tolerance}")
114         if s.recall < base_r - tolerance:
115             regressed = True
116             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
117         else:
118             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
119
120     return RegressionResult(
121         video_id=video_id, stage=stage, iou_threshold=iou_threshold,
122         precision=s.precision, recall=s.recall, f1=s.f1,
123         baseline_precision=base_p, baseline_recall=base_r,
124         tolerance=tolerance, regressed=regressed, reasons=reasons,
125     )
126
127
128 def regress_with_provider(db: Database, video_id: str, video_ref: str,
129                           tier: str = "file", stage: str = "final",
130                           iou_threshold: float = 0.5,
131                           detector: Optional[str] = None,
132                           tolerance: float = DEFAULT_TOLERANCE,
133                           baseline_path: str = DEFAULT_BASELINE_PATH,
134                           guideline: Optional[str] = None,
135                           sport: str = "badminton",
136                           provider_kwargs: Optional[dict] = None) -> RegressionResult:
137     """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
138
139     `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
140     once those providers land). `video_ref` is whatever that provider expects (a CSV
141     path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
142     predictions in the DB (the file MD5).
143     """
144     provider = annotation_registry.get(tier, *(provider_kwargs or {}))
145     gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
146     return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
147                   detector=detector, tolerance=tolerance, baseline_path=baseline_path)
148
149
150 def format_result(r: RegressionResult) -> str:
151     verdict = "REGRESSED" if r.regressed else "ok"
152     head = f"[{verdict}] {r.video_id} ({r.stage}) "
153     f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}"

```

```

154         if r.baseline_precision is not None:
155             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
156         return head + "".join(f"\n    - {why}" for why in r.reasons)
157

```

4. user (2026-06-10T06:11:24.570Z)

.github\workflows\ci.yml

5. user (2026-06-10T06:11:32.011Z)

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11    concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15    jobs:
16        # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
17        # module-level import error shipping while the suite stays green.
18        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
19        # on machines without the GPU stack (test_import_smoke stubs what's absent).
20        import-smoke:
21            name: Syntax sweep + import smoke (no GPU stack)
22            runs-on: ubuntu-latest
23            steps:
24                - uses: actions/checkout@v4
25                - uses: actions/setup-python@v5
26                  with:
27                      python-version: "3.13"
28                - name: Install minimal deps
29                  run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai
30                - name: Run import smoke tests
31                  run: python -m pytest tests/test_import_smoke.py -v
32
33        full-suite:
34            name: Full test suite (offline, mocked)
35            runs-on: ubuntu-latest
36            steps:
37                - uses: actions/checkout@v4
38                - uses: actions/setup-python@v5
39                  with:
40                      python-version: "3.13"
41                      cache: pip
42                - name: System libs for opencv-python
43                  run: |
44                      sudo apt-get update
45                      sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
46                - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
47                  run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
48                - name: Install requirements
49                  run: python -m pip install -r requirements.txt pytest
50                # obsreport (private, soft dep) is absent here: reporting tests skip via

```

```

51         # pytest.importorskip ? run_all_tests.py prints which.
52         - name: Run full suite
53         run: python run_all_tests.py
54

```

6. user (2026-06-10T06:11:33.570Z)

```

run_regression.py:10:    python run_regression.py --video-id <md5> --tier file --annotations
rallies.csv --update-baseline
run_regression.py:13:    python run_regression.py --video-id <md5> --tier file --annotations
rallies.csv
docs\CODE_MAP.md:74:Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py`
(manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
docs\CODE_MAP.md:90:- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit
codes semantic: **0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER
compare (snapshots even a regressing run); `:_default_db_path` resolves via
`backend.config.load_config`.
docs\CODE_MAP.md:282:| Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit
1=regress, 2=missing DB; snapshot via `--update-baseline` |
docs\CODE_MAP.md:313:| `test_eval_regression.py` | `eval_regression` baseline
store/compare/gate; also `@run_regression.py::main` (exit-code CI gate) |
docs\GOLDEN_SET_IMPLEMENTATION.md:8:[Omitted long matching line]
docs\GOLDEN_SET_IMPLEMENTATION.md:52:`regress(video, tier="fast", tolerance=0.05)`: get GT from
`annotation_registry.get(tier)`, read predictions via the existing `eval.harness`, score with
the existing `metrics`, compare against a per-video `baseline.json`, and flag a regression when
precision/recall drop beyond tolerance. Pure orchestration over existing parts + a small
baseline store. CLI: `python run_regression.py --video ... --tier file`.
docs\DEFERRED_HARDENING_PLAN.md:76: `run_regression.py` so the baseline is computed strictly on
the eval split.
tests\test_eval_regression.py:8:import run_regression
tests\test_eval_regression.py:101:    rc = run_regression.main(["--video-id", "vid1", "--tier",
"file",
tests\test_eval_regression.py:118:    rc = run_regression.main(["--video-id", "vid1", "--tier",
"file",
docs\archives\checkpoints\2026-06-eval-baseline\README.md:12:- Current regression baselines (in
output/ or tracked via run_regression.py).
docs\archives\checkpoints\2026-06-eval-baseline\README.md:22:- Impact of changes on the CI
regression gate (`run_regression.py`).
docs\archives\checkpoints\2026-06-eval-baseline\README.md:33:- Tools: run_eval.py,
run_phase3_eval.py, run_regression.py, backend/eval/.

```

7. user (2026-06-10T06:11:34.729Z)

```

64- -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
65- -> harness.evaluate -> EvalReport -> report_to_dict
66-     -> batch.aggregate (corpus micro/macro) @backend/eval/batch.py
67-     -> regression.regress(_with_provider) vs baseline @backend/eval/regression.py [CI
gate]
68-Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
69-Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier)
@backend/eval/{calibrate_local,distill_local}.py
70-Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
71-Gemini refinement driver (tuning, standalone):
gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py
72-Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
73-```
74:Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can
segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).

```

```

75-
76----
77-
--
87-- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
88-- `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
89-- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label);
config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed `AppConfig`
(same object the live pipeline feeds segmenters).
90-- `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
**0=ok, 1=REGRESSION (CI gate), 2=missing DB**. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run); `::default_db_path` resolves via
`backend.config.load_config`.
91-
92-- `@backend/config/__init__.py` ? ? canonical config import surface. Re-exports
`load_config`, `save_default_config` (from `.loader`) + `AppConfig`, `Config`, `IndexingConfig`,
`ValidationConfig` (from `models`); `__all__` = exactly those 6. Use `from backend.config
import load_config, AppConfig`.
93-- `@backend/config/models.py` ? Pydantic v2 models; **single source of truth for defaults**.
--
139-[Omitted long context line]
140-[Omitted long context line]
141-[Omitted long context line]
142:[Omitted long matching line]
143-
144-### 2.3a Audio (E1 probe; NOT adopted for fusion ? ADR-007)
145-[Omitted long context line]
--
279-| Run a video end-to-end | `@backend/main.py::main` (`--video-path` or `--server`) or
`@run_process_and_stitch.py::main` (edit hardcoded paths) |
280-| Score DB preds vs GT (one video) | `@run_eval.py::main` |
281-| Batch score a collector manifest | `@run_phase3_eval.py::main` (`--manifest`) |
282-| Guard a regression (CI) | `@run_regression.py::main` (`--video-id`); exit 1=regress,
2=missing DB; snapshot via `--update-baseline` |
283-| Run the whole test suite | `@run_all_tests.py` (repo root); or `pytest tests/` directly |
284-| Resume a crashed WASB pass | nothing to do ?
`@backend/pipeline/detectors/wasb_infer.py::run` auto-resumes from
`det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart |
285-| Add a validator | subclass `@backend/pipeline/validators/base.py:VideoValidator`;
`registry.register(instance)` in `@backend/pipeline/validators/__init__.py` (mind order;
producers before consumers) |
--
310-| `test_export_wasb_segments.py` | `eval.export_wasb_segments` segment/comparison CSV
schemas, mmss; export->slicer reuse |
311-| `test_eval_training.py` | `eval.training` label_candidates / feature extraction /
build_training_set |
312-| `test_eval_classifier.py` | `eval.classifier.LogisticRegression`
fit/predict_proba/balanced/JSON round-trip |
313-| `test_eval_regression.py` | `eval.regression` baseline store/compare/gate; also
`@run_regression.py::main` (exit-code CI gate) |
314-| `test_distill_local.py` | `eval.distill_local` build_candidates / predict_rallies /
learned-vs-baseline |
315-| `test_gemini_refine.py` | `eval.gemini_refine` merge_overlaps / _offset_segments (pure
helpers, no API) |
316-| `test_hit_detector.py` | `pipeline/audio/hit_detector` (detect_hits, onset_envelope,
_moving_stats) + `eval/audio/e1_probe.cluster_hits` |

```

```

1      # Byte-compiled / optimized / DLL files
2      __pycache__/
3      *.py[cod]
4      *$py.class
5      *.pt
6
7      # Local environment secrets
8      .env
9      *.env
10     .env.*
11
12     # C extensions
13     *.so
14
15     # Distribution / packaging
16     .Python
17     build/
18     develop-eggs/
19     dist/
20     downloads/
21     eggs/
22     .eggs/
23     lib/
24     lib64/
25     parts/
26     sdist/
27     var/
28     wheels/
29     share/python-wheels/
30     *.egg-info/
31     .installed.cfg
32     *.egg
33     MANIFEST
34
35     # SQLite databases & Local output files
36     *.db
37     *.db-journal
38     *.db-wal
39     output/
40     output_dir/
41     backend/static/highlights_*.mp4
42     backend/static/*_compiled.mp4
43
44     # OS generated files
45     .DS_Store
46     Thumbs.db
47     desktop.ini
48
49     # Coverage data
50     .coverage
51     .coverage.*
52     htmlcov/
53     .pytest_cache/
54
55     # Local Claude Code config (machine-specific)
56     .claude/
57     .clocovleerlaage
58     █
59     █

```

9. user (2026-06-10T06:11:43.634Z)

```

55     ## 2. Train/eval separation guardrails (debt #13)
56
57     ### Problem

```

```

58 Separation is by convention only. `backend/eval/calibrate_local.py` and
59 `calibrate_wasb.py` do honest leave-one-video-out (LOVO), but nothing enforces
60 that a video used to train the learned segmenter
61 (`backend/eval/training.py::build_training_set` ? `distill_local`) is absent from the
62 eval/regression set. Leakage would silently inflate the very metrics the product
63 story rests on (? in BACKLOG).
64
65 ### Approach
66 - Split unit = match, never clip (per `GOLDEN_SET_PHASE1_VIDEOS.md`: split by
67 match, hold ~? of each sport as eval).
68 - Add a `split: "train" | "eval"` field to the collector's `curate export`
69 manifest (cross-repo contract, ADR-002 lineage) ? or a separate eval manifest.
70 - New `backend/eval/splits.py`:
71   - `load_split(manifest) -> (train_ids, eval_ids)` keyed by match id (not just
72     file MD5 ? re-encoded duplicates share a match but differ in
73     `compute_video_id`, so group at match level).
74   - `assert_disjoint(train_ids, eval_ids)` ? raise on any overlap.
75 - Wire the guard into the training entrypoint (`distill_local`/`training`) and into
76   `run_regression.py` so the baseline is computed strictly on the eval split.
77
78 ### Risks
79 - Requires a collector-side manifest change (coordinate the cross-repo contract).
80 - Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
81   but the numbers will be noisy until the golden set grows.
82 - MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.
83
84 ### Verification
85 - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86 - Integration: a deliberately-leaked manifest fails the training entrypoint.
87
88 ---
89
90 ## 3. Async job model (debt #16)
91
92 ### Problem
93 `POST /api/process` runs validate ? segment ? report inline, blocking the
94 request for the full duration of a long video. Unworkable as a hosted/product API.
95
96 ### Approach (minimal first slice ? single self-hosted box)
97 - `POST /api/process` returns 202 + `job_id`; add `GET /api/jobs/{job_id}`
98   returning `{status, progress, result, reports}`.
99 - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`

```

10. assistant (2026-06-10T06:11:58.509Z)

The .gitignore line 39 (`output/`) and CI workflow claims check out. Now let me check whether the "not wired into CI" status is documented as a known deferral.

11. user (2026-06-10T06:12:01.008Z)

```

8:[Omitted long matching line]
34:| GS-2 | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
37:| GS-5 | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI, scheduled
run | GS-2, GS-4 | No |
49:- Design choice: `fetch()`/`annotate()` raise loudly on a missing/incomplete job rather
than returning `[]`, so a broken label source can't silently masquerade as "0 rallies" and skew
a regression to green.
52:`regress(video, tier="fast", tolerance=0.05)`: get GT from `annotation_registry.get(tier)`,
read predictions via the existing `eval.harness`, score with the existing `metrics`, compare
against a per-video `baseline.json`, and flag a regression when precision/recall drop beyond
tolerance. Pure orchestration over existing parts + a small baseline store. CLI: `python
run_regression.py --video ... --tier file`.
58:An independent strong labeler for the fast/CI tier. Must be a different model lineage than
whatever production uses (else shared blind spots ? false green). A fast-tier failure is a

```

smoke alarm that escalates to the GS-3 human tier, never an authoritative verdict on its own. 61:Wire `regress()` into CI (pre-merge gate on pipeline changes), an on-demand CLI (the "surprise regression"), and an optional scheduled run. Includes a deliberate re-snapshot path so legitimate production improvements can rebaseline instead of raising false regressions.

12. user (2026-06-10T06:12:02.093Z)

31-- ? **Internal gold key** ? hand-annotate 5?10 licensed clips to near-perfect as the vendor answer key. **Unblocked.** **Revisit:** anytime.
 32-- ? **GS-3 ? `HumanVendorProvider`** ? concrete adapter for the chosen annotation vendor (submit\poll\fetch + DPA\secure-link controls). **Vendor selected for the initial pilot: Roora** (Bengaluru; iMerit / Taskmonk / TELUS remain the fallback shortlist). **Revisit:** once the Roora pilot confirms quality, then build the adapter.
 33:- ? **GS-4 oracle model** ? `AutomatedProvider` interface is stubbed (this work) with a deterministic fake. Wire a *concrete* fast-tier auto-labeler later. **Oracle rule:** it MUST be a different model lineage than production, or the regression test is blind to shared failures. **Revisit:** when a production LLM step is chosen (pick the oracle's lineage to differ).
 34:- ? **GS-5 ? regression triggers** ? wire `regress()` into CI (pre-merge gate), on-demand CLI (the "surprise" case), and a scheduled run; include a deliberate re-baseline path. **Revisit:** after GS-4.
 35-- ? **Vendor pilot (Roora)** ? run the paid pilot with **Roora** as a **single-vendor pilot first** (not a multi-vendor bake-off): assess quality-per-rupee against the \$4 bar; escalate to the iMerit/Taskmonk fallback only if it misses. **Revisit:** the pilot doubles as calibration once the gold key exists.
 36-- ? **Phase-1 video collection** ? gather the round-1 golden corpus per `docs\GOLDEN\_SET\_PHASE1\_VIDEOS.md`: **static self-recorded** footage (mirror deployment, not broadcast), 3 sports incl. singles & doubles, across the diversity matrix; ~12 clips/sport with **? held out by match**. **Unblocked.**

13. user (2026-06-10T06:12:12.363Z)

```
1      # Golden-Set Subproject ? Design & Implementation Overview
2
3      **Status:** In progress . **Date:** 2026-05-30
4      **Companion docs:** design rationale + vendor shortlist in
[GOLDEN_SET_VENDORING.md](GOLDEN_SET_VENDORING.md) (ADR-006); scoring mechanics in
[EVALUATION.md](EVALUATION.md).
5
6      This is the **implementation tracker** for turning the golden-set/vendor *design* into
shipped code. It's split into small, independently-reviewable PRs. The whole subproject is
**vendor-agnostic by construction** ? only one slice (GS-3, `HumanVendorProvider`) depends on
which vendor you pick offline; everything else can land first.
7
8      > **Update (2026-06-02):** Vendor selected for the **initial pilot ? Roora**
(Bengaluru). The **annotation guideline, intake-manifest template, and NDA** are authored in the
collector repo `docs/annotation/`, and the collector's `import-local` ingestion now covers the
**3 pilot sports** (badminton, pickleball, table tennis). On the indexer side, GS-1
(`file_provider.py`), GS-2 (`backend/eval/regression.py` + `run_regression.py`), and the GS-4
oracle scaffold (`automated_provider.py`) have landed; the remaining slice is the vendor-
dependent **GS-3 `HumanVendorProvider`**, parked until the Roora pilot confirms quality.
9
10     ---
11
12     ## Architecture in one picture
13
14     ```
15         video_ref ??? AnnotationProvider.submit()/poll()/fetch() ???
List[(start,end)]  ground truth
16
17         ?           ?           ?
FileProvider  HumanVendor  Automated(oracle)
18         (GS-1)      (GS-3)      (GS-4)
backend.eval.metrics.score() ??? existing, unchanged
19
20         production pipeline ??? DB ??? backend.eval.harness.get_predictions() ???
List[(start,end)]  predictions
```

```

21                                                     ?
22 regress(video, tier) (GS-2)
compares the two
23     ``
24
25     **The keystone:** a provider's `fetch()` returns the **same `List[(start, end)]`** that
`eval.annotations.load_annotations()` already returns and `eval.metrics.score()` already
consumes. So one scorer grades all three of: product-vs-GT, vendor-vs-answer-key, and annotator-
vs-annotator (IAA). No new scoring code ? only new *sources* of intervals.
26
27     ---
28
29     ## PR breakdown
30
31     | Slice | Branch | Scope | Depends on | Vendor-dependent? |
32     |---|---|---|---|---|
33     | **GS-1** | `feat/annotation-provider-registry` | `AnnotationProvider` ABC + registry +
`FileProvider` + tests | ? | No |
34     | **GS-2** | `feat/regression-runner` | `regress(video, tier, tolerance)` over existing
harness + `baseline.json` snapshot/compare + CLI | GS-1 | No |
35     | **GS-3** | `feat/human-vendor-provider` | `HumanVendorProvider` (submit/poll/fetch
against the chosen vendor's API or file-drop), DPA/secure-link handling | GS-1, **your vendor
pick** | **Yes** |
36     | **GS-4** | `feat/automated-oracle-provider` | `AutomatedProvider` fast-tier oracle ?
**interface stub** (injected labeler + fake; concrete model = backlog) | GS-1, GS-2 | No |
37     | **GS-5** | `feat/regression-triggers` | Wire triggers: CI gate, on-demand CLI,
scheduled run | GS-2, GS-4 | No |
38
39     Each row is one PR. GS-1 ? GS-2 ? GS-4 ? GS-5 can proceed **now**, independent of vendor
selection. GS-3 is parked until you've chosen a vendor (then it's a thin adapter behind the GS-1
interface).
40
41     ---
42
43     ## Slice details
44
45     ### GS-1 ? Provider registry + FileProvider ? (this PR)

```

14. user (2026-06-10T06:13:04.064Z)

Structured output provided successfully

15. assistant (2026-06-10T06:13:10.611Z)

Verification complete ? verdict returned via structured output.